CO1 ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

CSA02 – C Programming | Grama Jyothi Billing Program | 50 Marks

Name: Prithiga S Register No: 1965260110

GitHub: https://github.com/prithiga120808-rgb/CSA0232

Task 1 – Problem Analysis

4 marks

- Inputs: house no., category, previous and current reading (0–65535); outputs: units, energy charge, fixed charge, total
- Constants: slab limits 100 / 200 units, rates 0 / 225 / 450 paise, fixed charge 5000 paise
- Negative: raw difference after roll-over; fractional: Rs 2.25 rates; very large: district total
- Rules: bill never negative; total = fixed + energy; units = (current − previous) mod 65536
- Assumption: one roll-over at most, accepted only if previous ≥ 60000

```
Text vs integer (measured)
4,000,000 readings, gcc -O0
integer subtract  : 2.3 ns
strtol + subtract : 26.4 ns
strcmp compare    : 4.3 ns

bytes per reading
unsigned short = 2
text "01530"   = 6
```

Alternative & complexity: Readings as text ("01530") need O(d) conversion and compare, and d+1 bytes (6 vs 2). Measured about 11x slower to convert and subtract, so integers are chosen.

Task 2 – Data Types

4 marks

- meter_t = unsigned short (2 bytes, 0–65535) matches the 16-bit register; units unsigned int
- Money in long paise, district total in long long; never float rupees
- float fails: 4,000 × Rs 123.45 gives 493,781.22 instead of 493,800.00
- Why 256: 120 − 65400 = −65280 as int; converted to unsigned short it becomes −65280 + 65536 = 256

```
TODO 1 and output
return (meter_t)(current - previous);

sizeof(meter_t) = 2 bytes, max = 65535
Consumption: 330 units
Consumption: 256 units
```

Alternative & complexity: Money as double rupees: round once per bill, error grows about O(n·ε) versus exactly 0 for paise. Time O(1) both; space 8 bytes (double, long) vs 4 (float).

Task 3 – Constants

3 marks

- #define for tariff values: no storage, no type; const: typed, visible in the debugger; enum for related integer codes
- Naming: UPPER_CASE with unit suffix (_PAISE, _UNITS)
- A tariff revision is made in one place only
- RATE2_PAISE 225 → 230 changes every slab-2 bill consistently

```
TODO 1 (complete)
if (units > SLAB2_UNITS) {
 paise += (long)(units-SLAB2_UNITS)
         * RATE3_PAISE;
 units = SLAB2_UNITS; }
if (units > SLAB1_UNITS) {
 paise += (long)(units-SLAB1_UNITS)
         * RATE2_PAISE;
 units = SLAB1_UNITS; }

350 units -> Rs 950.00
Rate 230: 150 units -> Rs 165.00
```

Alternative & complexity: const or enum: same code, adds type checking and debugger names; a #define uses no storage, a const may. Running time stays O(1) either way.

Task 4 – Macros

4 marks

- gcc -E shows macros are plain text replacement
- SQUARE(x) as x*x gives 9 for a+1 (a = 4); ((x)*(x)) gives 25
- MAX(i++, j) evaluates i++ twice, leaving i = 7
- Use a function when arguments have side effects or type safety matters

```
gcc -E evidence
a+1*a+1          -> 9 (wrong)
((a+1)*(a+1))    -> 25

(((i++)>(j))?(i++):(j))
MAX(i++,j)     = 6, i = 7
max_fn(i++,j)  = 5, i = 6
```

Alternative & complexity: static inline functions: same O(1) time when inlined, each argument evaluated once. A macro is copied at every use, so code size is O(k) for k uses.

Task 5 – Headers and Conditional Compilation

3 marks

- Structure: includes, global declarations and prototypes, main(), other functions
- Include guards stop double inclusion and redefinition errors
- #include <file> searches system directories; "file" searches the local directory first
- #ifdef DEBUG prints only when compiled with -DDEBUG

```
Header and DEBUG mode
#ifndef BILLING_H
#define BILLING_H
...
#endif

gcc -DDEBUG -o debug main.c billing.c
[DEBUG] consumption: prev=65400
  curr=120 units=256

text size 1777 B -> 1947 B (+170)
```

Alternative & complexity: if (debug) costs one test per call, O(1), and the debug code always stays in the executable. #ifdef costs nothing in the normal build but needs recompiling.

Task 6 – Formatted Input and Output

4 marks

- Specifiers: %d %u %ld %lld %f %lf %c %s
- %d with a long 5000000000 printed 705032704; gcc -Wformat warned
- Check scanf returns the expected count to catch letters
- Print paise as Rs p/100 . p%100 using %02ld

```
Validation
"102 abc 1530" -> scanf returned 1
"104 1200"     -> scanf returned 2
"101 1200 1530" -> returned 3

printf("Rs %ld.%02ld",
       p/100, p%100);
16250 -> Rs 162.50
```

Alternative & complexity: fgets + sscanf: O(L) time and an L+2 byte buffer; a bad line is already consumed, so recovery is easy. scanf leaves bad characters in the input.

Task 7 – Type Conversion and Precision

4 marks

- Conversion points: promotion in current – previous, int to long before multiplying, p / 100 integer division
- Surprises: 250/100 = 2; (int)5000000000L = 705032704; −10 < 5u is false
- Rounding rule: round half up, applied once at the end

```
Three+ surprises
250/100            = 2
(int)5000000000L   = 705032704
-10 < 5u           -> 0
1200-1530 = -330, as ushort 65206
int*int*int  = 2050327040
(long) first = 45000000000
```

Alternative & complexity: All float, rounded once: error −Rs 826 at 60,000 bills and +Rs 916,507 at 800,000; paise stay exact. Both O(1) per add; float 4 bytes, long 8.

Task 8 – Compilation Process

4 marks

- Preprocessor expanded headers and macros and removed comments
- Compiler produced assembly; assembler produced main.o and billing.o; linker built app
- Compile, link and run-time errors each demonstrated
- Warnings fixed: -Wsign-compare, -Wformat, -Wuninitialized

```
Stages and errors
gcc -E  main.c: 10 -> 828 lines
gcc -S  main.s: 87 lines
main.o 1800 B, billing.o 1456 B
app 16072 B

compile: expected expression
link: undefined reference to
  `compute_bill`
run: Floating point exception
```

Alternative & complexity: Separate .c files: after one change only that file is recompiled plus a link, instead of the whole program. Costs several object files in extra space.

Task 9 – Implementation and Testing

4 marks

- Tested normal, boundary (0, 100, 200), roll-over, and invalid inputs
- Rejected: letters, negative reading, current < previous without roll-over
- Hand-calculated 5-bill set total matches the program: Rs 1487.50

```
Test results
0 units      Rs 50.00
100 units    Rs 50.00
150 units    Rs 162.50
200 units    Rs 275.00
65400->120   Rs 527.00 (256)
350 units    Rs 950.00
201 units    Rs 279.50
letters / -5 / 1530->1200: rejected
5 bills = Rs 1487.50
```

Alternative & complexity: Slabs in arrays with a loop: O(s) time (6.8 ns vs 2.9 ns measured) and O(s) extra space, but a new slab is only a table row, so it is easier to change.

Task 10 – Expectation vs Execution

4 marks

- At 4,000 bills every type is exact
- A 32-bit int total overflows by 60,000 bills; float drifts
- long and long long are exact at district scale
- int 4 B, long 8 B (LP64), long long 8 B; the standard guarantees only minimums

```
Measured totals
n         exact paise      int32      float diff
4,000     248,900,825      OK         0.00
60,000    3,693,505,800    OVERFLOW   -826.00
800,000   49,050,067,675   OVERFLOW   +916,507.25
```

Alternative & complexity: Total in double: exact for integers up to 2^53 (about 9.0e15), while long long overflows only at 9.2e18. Addition 0.63 ns (long long) vs 2.39 ns (double); both 8 bytes.

Task 11 – Final Recommendation

4 marks

- meter_t unsigned short, units unsigned int, bill long paise, totals long long
- #define tariff in one header; inline functions; include guards and #ifdef DEBUG; fgets + sscanf
- Limits: roll-over rule is assumed; long is 4 bytes on Windows; three fixed slabs
- Correct at district scale: max bill about 29.5 million paise, total 4.9e10 paise, far below 9.2e18

```
Revision procedure
Tariff revision
1. edit #define in billing.h
2. recompile
3. rerun tests
4. redistribute program
```

Alternative & complexity: Tariff from a config file: one-time O(k) start-up read (k = 9) and about 70 bytes, no recompiling; the file must be validated.

Task 12 (A, B) – Impact and Evaluation

8 marks

- Changed: Wh readings every 15 min (35,040 per house a year), peak 18:00–22:00 costs 20% more, tariff editable without recompiling
- A 16-bit cumulative Wh register wraps at 65.5 kWh; counters and totals need 64-bit
- Tariff values and peak hours can no longer be #define; WH_PER_KWH, SLOTS_PER_DAY stay

```
Evaluation
Largest values (10 kW house)
per reading     2,500 Wh
house-year      87,600,000 Wh
district-year   7.0e13 Wh
readings/year   28,032,000,000
Wh x rate x pct 3.9e11 (> int)
```

Alternative & complexity: Decision: MODIFY. Keep paise integers, headers and validation; widen to int64_t; move the tariff to a file. No full redesign is needed.

Task 12 (C, D) – Adaptation and Defence

8 marks

- tariff_t loaded from tariff.cfg (all 9 keys checked); each reading charged on its slab, peak × 120%
- Charge kept in 1e-5 paise and rounded once per month
- 288 kWh month: Rs 692.24 (Rs 671.00 without the peak rule); an independent Python check agrees
- PEAK_PERCENT 130 gives Rs 702.86 with no recompile

```
Working result
month: 2880 readings, 288 kWh
energy Rs 642.24
fixed  Rs 50.00
TOTAL  Rs 692.24
python check: Rs 692.24
PEAK_PERCENT=130 -> Rs 702.86
```

Alternative & complexity: Defence: the largest values need 64-bit; a run-time tariff adds a file read and validation; the tested paise core is kept, so a redesign is not justified.